

EMPLOYEE PAYROLL MANAGEMENT SYSTEM

23CS46C - JAVA PROGRAMMING LABORATORY

Submitted by

SUBIKSHAN M 2212110

MOHAMMED ASEEL M 2312028

In partial fulfillment for the award of the degree

of

BACHELOR OF ENGINEERING

in

COMPUTER SCIENCE AND ENGINEERING



NATIONAL ENGINEERING COLLEGE

(An Autonomous Institution affiliated to Anna University, Chennai)

K.R.NAGAR, KOVILPATTI - 628503

APRIL - 2025

NATIONAL ENGINEERING COLLEGE
(An Autonomous Institution affiliated to Anna University, Chennai)
K.R.NAGAR, KOVILPATTI - 628503



BONAFIDE CERTIFICATE

This is to certify that this project report, “Employee Payroll Management System”, is the bonafide work of **SUBIKSHAN M (2212110) , MOHAMMED ASEEL M (2312028)**, who carried out the project work under my supervision.

Course Instructor/Guide

Submitted to the **23CS46C - JAVA PROGRAMMING LABORATORY** Viva-Voce examination held at National Engineering College, K.R.Nagar, Kovilpatti on _____

Internal Examiner

Co Examiner

TABLE OF CONTENTS

CH NO	TITLE	PAGE NO
1	INTRODUCTION	3
2	OBJECTIVES	4
3	DESCRIPTION	5
4	CLASS DIAGRAM	6
5	TABLE STRUCTURES	7
6	ER DIAGRAM	8
7	MODULES	9
8	CODING	12
9	SCREENSHOTS	61
10	CONCLUSION	69

CHAPTER 1

INTRODUCTION

The Employee Payroll Management System is a web-based application developed to streamline and automate the process of managing employee salary records within an organization. This system eliminates the need for manual payroll calculation and record-keeping, ensuring accuracy, efficiency, and transparency in handling payroll operations.

The system supports two types of users: **Admin** and **Employee**, each having distinct access and functionalities. The **Admin** is responsible for managing employee records, generating and updating payroll information, processing salary details, and maintaining payroll history. Admins can view, add, edit, or delete employee and payroll entries as required.

The **Employee** can log in to view their personal payroll details, including base salary, bonuses, deductions, and net payable amounts. This ensures that employees are kept informed and have access to their financial records in a secure and transparent manner. The system provides features such as monthly payroll generation, payroll status tracking, historical records, and department-based filtering. By automating calculations and maintaining structured data, the system reduces human error, enhances data security, and saves valuable administrative time.

This user-friendly and secure platform is suitable for businesses, educational institutions, and any organization seeking an organized approach to managing employee salaries and financial records.

CHAPTER 2

OBJECTIVES

The proposed payroll management system is designed to provide distinct and secure login access for both Admin and Employee users, ensuring role-based functionality.

Admins will have full control over employee records, with the ability to add, update, and delete information as needed. They will also manage payroll operations by creating, viewing, and updating payroll details for each employee. The system will automate salary calculations, including components such as base pay, bonuses, deductions, and the final net salary, minimizing human error and improving accuracy.

Employees will be able to securely log in to view their personal payroll details and full payment history, promoting transparency and trust. By digitizing and centralizing all payroll processes, the system enhances administrative efficiency, reduces paperwork, and ensures easy access to accurate and up-to-date payroll data for both Admins and Employees.

CHAPTER 3

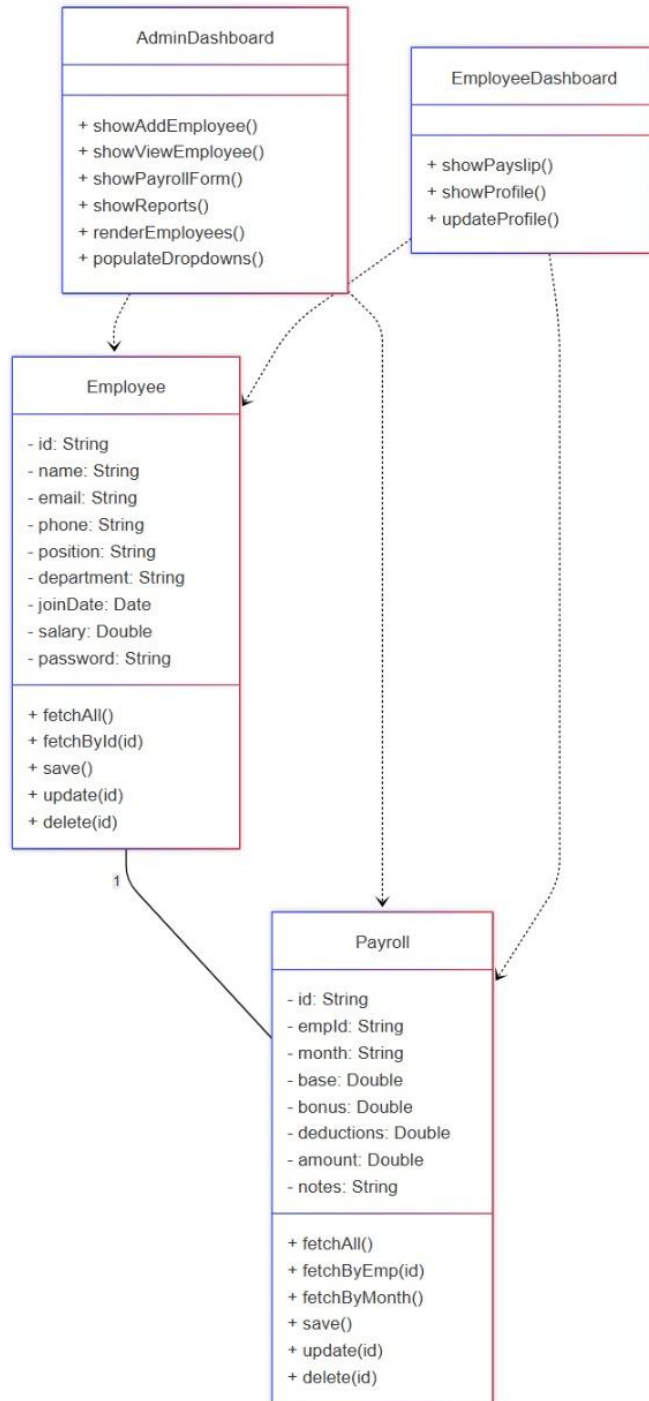
DESCRIPTION

The Employee Payroll Management System is a web-based application designed to efficiently manage employee salary and payroll details within an organization. The system includes two types of users: **Admin** and **Employee**. The **Admin** has the ability to add, update, and delete employee records, as well as create and manage monthly payroll data, including salary, bonuses, deductions, and net pay. All payroll information is securely stored in a database for easy tracking and reporting.

The **Employee** can log in to view their personal payroll details, including salary breakdown and payment history. The system ensures secure access through a login page and provides a user-friendly interface for both Admin and Employee users. This project helps reduce manual calculations, improves accuracy, enhances data security, and maintains well-organized digital payroll records for the organization.

CHAPTER 4

CLASS DIAGRAM



CHAPTER 5

TABLE STRUCTURES

EMPLOYEE DETAILS

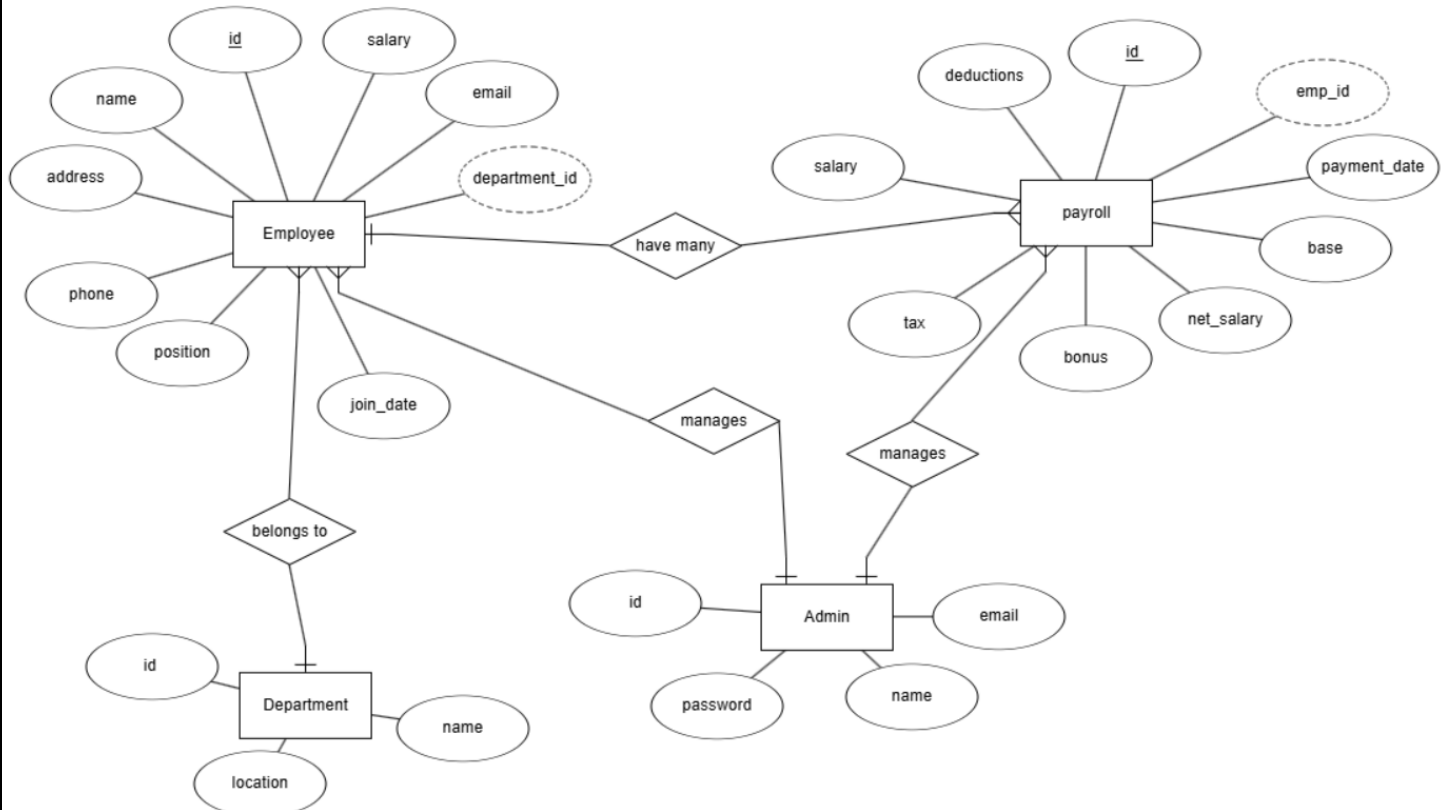
Result Grid									
		Filter Rows:			Edit:			Export/Import:	Wrap Cell Content: IA
	id	created_at	department	email	join_date	name	password	phone	position
▶	212	2025-05-10 06:01:27.423936	IT	sung@gmail.com	2024-12-16	Sung jin woo	employee123	9334238419	HR
	28	2025-05-10 05:56:09.334765	IT	pradeep@gmail.com	2025-05-08	pradeep	employee123	3984529421	Clerk
	EMP001	2025-05-10 05:54:22.633687	IT	john@example.com	2023-01-15	John Doe	employee123	1234567890	Manager
	EMP002	2025-05-10 05:54:22.570772	Human Resources	jane@example.com	2023-02-10	Jane Smith	employee123	9876543210	HR
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

PAYROLLS

Result Grid											
		Filter Rows:			Edit:			Export/Import:	Wrap Cell Content: IA		
	id	amount	base	base_salary	bonus	created_at	created_date	deductions	month	notes	emp_id
▶	1	11	5000	5000	10	2025-05-10	2025-05-10	4999	2025-05	nothing personal...	28
	2	110000	100000	100000	10000	2025-05-10	2025-05-10	0	2025-05		212
	3	80000	75000	75000	5000	2025-05-10	2025-05-10	0	2025-05		EMP001
	4	69000	65000	65000	4000	2025-05-10	2025-05-10	0	2025-05		EMP002
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

CHAPTER 6

ER DIAGRAM



CHAPTER 7

MODULES

7.1. Authentication Module

- Allows separate login for Admin and Employee users.
- Uses secure authentication with username and password.
- Prevents unauthorized access to sensitive payroll data

7.2. Admin Dashboard Module

- Admin can add, update, and delete employee records.
- Enables the creation and management of payroll details for each employee.
- Displays a list of all employees and their payroll history.
- Provides full administrative control over the payroll system.

7.3. Employee Dashboard Module

- Employees can view their own payroll details.
- Shows breakdown of salary, bonuses, deductions, and net pay.
- Provides access to historical payroll records for transparency.

7.4. Payroll Management Module

- Automates salary computation based on input components.
- Admin can generate payroll per employee per month.
- Ensures consistency and accuracy in salary calculation.
- Prevents duplication by checking existing payroll entries.

7.5. Database Management Module

- Stores employee information, payroll data, and user credentials.
- Ensures data integrity and security with proper schema relationships.
- Supports efficient data queries, updates, and retrieval.

6. Frontend User Interface Module

- Provides a user-friendly interface for Admin and Employee roles.
- Includes forms for employee management and payroll entry.
- Ensures intuitive navigation and ease of use.

7. Session Management Module

- Manages secure login and logout functionality.
- Protects session data and restricts access after logout.
- Maintains active sessions only during authenticated usage.

CHAPTER 8

CODING

Employee Model

```
package com.example.demopayrollpro.model;

import java.time.LocalDateTime;

import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.Id;
import jakarta.persistence.PrePersist;
import jakarta.persistence.Table;

@Entity
@Table(name = "employees")
```

```

public class Employee {

    @Id
    @Column(columnDefinition = "VARCHAR(255)")
    private String id;

    private String name;
    private String email;
    private String phone;
    private String position;
    private String department;
    private String joinDate;
    private double salary;
    private String password;

    @Column(name = "created_at", nullable = false, updatable = false)
    private LocalDateTime createdAt;

    @PrePersist
    protected void onCreate() {
        this.createdAt = LocalDateTime.now();
    }

    public Employee() {
    }

    public Employee(String id, String name, String email, String phone, String position,
        String department, String joinDate, double salary, String password) {
        this.id = id;
        this.name = name;
        this.email = email;
        this.phone = phone;
        this.position = position;
        this.department = department;
        this.joinDate = joinDate;
        this.salary = salary;
        this.password = password;
    }

    public String getId() {
        return id;
    }

    public void setId(String id) {
        this.id = id;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {

```

```
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public String getPhone() {
        return phone;
    }

    public void setPhone(String phone) {
        this.phone = phone;
    }

    public String getPosition() {
        return position;
    }

    public void setPosition(String position) {
        this.position = position;
    }

    public String getDepartment() {
        return department;
    }

    public void setDepartment(String department) {
        this.department = department;
    }

    public String getJoinDate() {
        return joinDate;
    }

    public void setJoinDate(String joinDate) {
        this.joinDate = joinDate;
    }

    public double getSalary() {
        return salary;
    }

    public void setSalary(double salary) {
        this.salary = salary;
    }

    public String getPassword() {
        return password;
    }
}
```

```

    }

    public void setPassword(String password) {
        this.password = password;
    }
    public LocalDateTime getCreatedAt() {
        return createdAt;
    }

    public void setCreatedAt(LocalDateTime createdAt) {
        this.createdAt = createdAt;
    }

    @Override
    public String toString() {
        return "Employee{" +
            "id=" + id + "\" +
            ", name=" + name + "\" +
            ", position=" + position + "\" +
            ", department=" + department + "\" +
            ", salary=" + salary +
            "}";
    }
}

```

EMPLOYEE CONTROLLER

```

package com.example.demopayrollpro.controller;

import java.util.HashMap;
import java.util.List;
import java.util.Map;
import java.util.Optional;

import org.slf4j.Logger;
import org.slf4j.LoggerFactory;
import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;
import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.example.demopayrollpro.model.Employee;
import com.example.demopayrollpro.service.EmployeeService;

@RestController
@RequestMapping("/api/employees")
@CrossOrigin(origins = "*")

```

```

public class EmployeeController {

    private static final Logger logger = LoggerFactory.getLogger(EmployeeController.class);

    @Autowired
    private EmployeeService employeeService;

    @GetMapping
    public ResponseEntity<List<Employee>> getAllEmployees() {
        logger.info("Fetching all employees");
        List<Employee> employees = employeeService.getAllEmployees();
        return ResponseEntity.ok(employees);
    }

    @GetMapping("/{id}")
    public ResponseEntity<Employee> getEmployeeById(@PathVariable String id) {
        logger.info("Fetching employee with ID: {}", id);
        try {
            // No need to convert String to Long anymore
            Optional<Employee> employee = employeeService.getEmployeeById(id);
            return employee.map(ResponseEntity::ok)
                .orElseGet(() -> {
                    logger.warn("Employee with ID: {} not found", id);
                    return ResponseEntity.notFound().build();
                });
        } catch (Exception e) {
            logger.error("Error fetching employee: {}", e.getMessage(), e);
            return ResponseEntity.badRequest().build();
        }
    }

    @PostMapping
    public ResponseEntity<?> createEmployee(@RequestBody Employee employee) {
        try {
            logger.info("Creating new employee with ID: {}", employee.getId());

            if (employeeService.employeeExists(employee.getId())) {
                logger.warn("Employee ID: {} already exists", employee.getId());
                return ResponseEntity.status(HttpStatus.CONFLICT)
                    .body(Map.of("error", "Employee ID already exists"));
            }

            if (employee.getPassword() == null || employee.getPassword().trim().isEmpty()) {
                employee.setPassword("employee123");
            }

            if (employee.getName() == null || employee.getName().trim().isEmpty()) {
                return ResponseEntity.badRequest().body(Map.of("error", "Employee name is required"));
            }

            Employee savedEmployee = employeeService.saveEmployee(employee);
            logger.info("Employee created successfully: {}", savedEmployee.getId());
        }
    }
}

```

```

        return ResponseEntity.status(HttpStatus.CREATED).body(savedEmployee);
    } catch (Exception e) {
        logger.error("Error creating employee: {}", e.getMessage(), e);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of("error", "Failed to create employee: " + e.getMessage()));
    }
}

@PutMapping("/{id}")
public ResponseEntity<?> updateEmployee(@PathVariable String id, @RequestBody
Employee employee) {
    try {
        logger.info("Updating employee with ID: {}", id);

        if (!employeeService.employeeExists(id)) {
            logger.warn("Employee with ID: {} not found for update", id);
            return ResponseEntity.notFound().build();
        }

        employee.setId(id);

        if (employee.getPassword() == null || employee.getPassword().trim().isEmpty()) {
            employeeService.getEmployeeById(id).ifPresent(
                existingEmployee -> employee.setPassword(existingEmployee.getPassword())
            );
        }

        Employee updatedEmployee = employeeService.saveEmployee(employee);
        logger.info("Employee updated successfully: {}", updatedEmployee.getId());
        return ResponseEntity.ok(updatedEmployee);
    } catch (Exception e) {
        logger.error("Error updating employee: {}", e.getMessage(), e);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of("error", "Failed to update employee: " + e.getMessage()));
    }
}

>DeleteMapping("/{id}")
public ResponseEntity<?> deleteEmployee(@PathVariable String id) {
    try {
        // No need to convert String to Long anymore
        logger.info("Deleting employee with ID: {}", id);

        if (!employeeService.employeeExists(id)) {
            logger.warn("Employee with ID: {} not found for deletion", id);
            return ResponseEntity.notFound().build();
        }

        employeeService.deleteEmployee(id);
        logger.info("Employee deleted successfully: {}", id);
        return ResponseEntity.noContent().build();
    } catch (Exception e) {
        logger.error("Error deleting employee: {}", e.getMessage(), e);

```

```

        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of("error", "Failed to delete employee: " + e.getMessage()));
    }
}

```

```

@PostMapping("/authenticate")

```

```

public ResponseEntity<?> authenticateEmployee(
    @RequestParam String id,
    @RequestParam String password) {

```

```

    try {
        // No need to convert String to Long anymore
        logger.info("Authenticating employee with ID: {}", id);

```

```

        Optional<Employee> employeeOpt = employeeService.authenticateEmployee(id,
password);

```

```

        if (employeeOpt.isPresent()) {
            Employee employee = employeeOpt.get();

```

```

            Map<String, Object> response = new HashMap<>();
            response.put("id", employee.getId());
            response.put("name", employee.getName());
            response.put("email", employee.getEmail());
            response.put("phone", employee.getPhone());
            response.put("position", employee.getPosition());
            response.put("department", employee.getDepartment());
            response.put("joinDate", employee.getJoinDate());
            response.put("salary", employee.getSalary());

```

```

            logger.info("Employee authenticated successfully: {}", id);
            return ResponseEntity.ok(response);
        }

```

```

        logger.warn("Authentication failed for employee ID: {}", id);
        return ResponseEntity.status(HttpStatus.UNAUTHORIZED)
            .body(Map.of("error", "Invalid credentials"));
    } catch (Exception e) {

```

```

        logger.error("Error during authentication: {}", e.getMessage(), e);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of("error", "Authentication error: " + e.getMessage()));
    }
}

```

```

@GetMapping("/search")

```

```

public ResponseEntity<?> searchEmployees(@RequestParam String query) {

```

```

    try {
        logger.info("Searching employees with query: {}", query);
        List<Employee> matchingEmployees = employeeService.searchEmployees(query);
        return ResponseEntity.ok(matchingEmployees);
    } catch (Exception e) {
        logger.error("Error searching employees: {}", e.getMessage(), e);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)

```

```

        .body(Map.of("error", "Error searching employees: " + e.getMessage()));
    }
}

@GetMapping("/department/{department}")
public ResponseEntity<?> getEmployeesByDepartment(@PathVariable String department) {
    try {
        logger.info("Fetching employees by department: {}", department);
        List<Employee> employees =
employeeService.getEmployeesByDepartment(department);
        return ResponseEntity.ok(employees);
    } catch (Exception e) {
        logger.error("Error fetching employees by department: {}", e.getMessage(), e);
        return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
            .body(Map.of("error", "Error fetching employees by department: " +
e.getMessage()));
    }
}

@GetMapping("/check-id/{id}")
public ResponseEntity<?> checkIdAvailability(@PathVariable String id) {
    try {
        // No need to convert String to Long anymore
        logger.info("Checking availability of employee ID: {}", id);
        boolean isAvailable = !employeeService.employeeExists(id);
        Map<String, Boolean> response = new HashMap<>();
        response.put("available", isAvailable);
        return ResponseEntity.ok(response);
    } catch (Exception e) {
        logger.error("Error checking ID availability: {}", e.getMessage(), e);
        return ResponseEntity.badRequest().body(Map.of("error", "Error checking ID"));
    }
}
}

```

EMPLOYEE SERVICE

```

package com.example.demopayrollpro.service;

import java.util.ArrayList;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.example.demopayrollpro.model.Employee;
import com.example.demopayrollpro.repository.EmployeeRepository;

@Service
public class EmployeeService {

```

```

@Autowired
private EmployeeRepository employeeRepository;

public List<Employee> getAllEmployees() {
    return employeeRepository.findAll();
}

public Optional<Employee> getEmployeeById(String id) { // Changed from Long to String
    return employeeRepository.findById(id);
}

public Employee saveEmployee(Employee employee) {
    return employeeRepository.save(employee);
}

public void deleteEmployee(String id) { // Changed from Long to String
    employeeRepository.deleteById(id);
}

public boolean employeeExists(String id) { // Changed from Long to String
    return employeeRepository.existsById(id);
}

public Optional<Employee> authenticateEmployee(String id, String password) { // Changed
from Long to String
    Optional<Employee> employeeOpt = employeeRepository.findById(id);

    if (employeeOpt.isPresent() && employeeOpt.get().getPassword().equals(password)) {
        return employeeOpt;
    }

    return Optional.empty();
}

public List<Employee> searchEmployees(String query) {
    if (query == null || query.trim().isEmpty()) {
        return getAllEmployees();
    }

    String lowercaseQuery = query.toLowerCase().trim();

    return employeeRepository.findAll().stream()
        .filter(employee ->
            (employee.getName() != null &&
employee.getName().toLowerCase().contains(lowercaseQuery)) ||
            (employee.getDepartment() != null &&
employee.getDepartment().toLowerCase().contains(lowercaseQuery)) ||
            (employee.getPosition() != null &&
employee.getPosition().toLowerCase().contains(lowercaseQuery))
        )
        .collect(Collectors.toList());
}

```

```

public List<Employee> getEmployeesByDepartment(String department) {
    if (department == null || department.trim().isEmpty()) {
        return new ArrayList<>();
    }

    String lowercaseDepartment = department.toLowerCase().trim();

    return employeeRepository.findAll().stream()
        .filter(employee ->
            employee.getDepartment() != null &&
            employee.getDepartment().toLowerCase().equals(lowercaseDepartment)
        )
        .collect(Collectors.toList());
}
}

```

EMPLOYEE REPOSITORY

```

package com.example.demopayrollpro.repository;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.example.demopayrollpro.model.Employee;

@Repository
public interface EmployeeRepository extends JpaRepository<Employee, String> { // Changed
    from Long to String
    // Custom query methods can be added here
}

```

YEARMONTHCONVERTER MODEL

```

package com.example.demopayrollpro.converter;

import jakarta.persistence.AttributeConverter;
import jakarta.persistence.Converter;
import java.time.YearMonth;

@Converter(autoApply = false)
public class YearMonthConverter implements AttributeConverter<YearMonth, String> {
    @Override
    public String convertToDatabaseColumn(YearMonth attribute) {
        return (attribute != null) ? attribute.toString() : null;
    }
}

```

```

@Override
public YearMonth convertToEntityAttribute(String dbData) {
    return (dbData != null && !dbData.isEmpty()) ? YearMonth.parse(dbData) : null;
}
}

```

PAYROLL MODEL

```

package com.example.demopayrollpro.model;

import java.time.LocalDate;
import java.time.YearMonth;

import com.example.demopayrollpro.converter.YearMonthConverter;

import jakarta.persistence.Column;
import jakarta.persistence.Convert;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.JoinColumn;
import jakarta.persistence.ManyToOne;
import jakarta.persistence.Table;
import lombok.AllArgsConstructor;
import lombok.Data;
import lombok.NoArgsConstructor;

@Entity
@Table(name = "payrolls")
@Data
@NoArgsConstructor
@AllArgsConstructor
public class Payroll {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @ManyToOne
    @JoinColumn(name = "emp_id", nullable = false)
    private Employee employee;

    @Convert(converter = YearMonthConverter.class)
    @Column(nullable = false)
    private YearMonth month;

    // Main field used in the entity
    @Column(name = "base")

```

```

private double base;

// Synchronized field for database compatibility
@Column(name = "base_salary")
private double baseSalary;

private double bonus;
private double deductions;
private double amount;
private String notes;

// Changed column name to match database schema
@Column(name = "created_at", nullable = false)
private LocalDate createdDate;

// Keep created_date field for backward compatibility (if needed)
@Column(name = "created_date")
private LocalDate createdDateLegacy;

// Override the setBase method to ensure both fields are set
public void setBase(double base) {
    this.base = base;
    this.baseSalary = base;
}

// Override the setBaseSalary method to ensure both fields are set
public void setBaseSalary(double baseSalary) {
    this.baseSalary = baseSalary;
    this.base = baseSalary;
}

// Update createdDate setter to sync both date fields
public void setCreatedDate(LocalDate date) {
    this.createdDate = date;
    this.createdDateLegacy = date;
}
}

```

PAYROLL CONTROLLER

```

package com.example.demopayrollpro.controller;

import java.time.LocalDate;
import java.time.YearMonth;
import java.time.format.DateTimeParseException;
import java.util.List;
import java.util.Map;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.http.HttpStatus;

```

```

import org.springframework.http.ResponseEntity;
import org.springframework.web.bind.annotation.*;

import com.example.demopayrollpro.model.Employee;
import com.example.demopayrollpro.model.Payroll;
import com.example.demopayrollpro.service.EmployeeService;
import com.example.demopayrollpro.service.PayrollService;

@RestController
@RequestMapping("/api/payrolls")
@CrossOrigin(origins = "*")
public class PayrollController {

    @Autowired
    private PayrollService payrollService;

    @Autowired
    private EmployeeService employeeService;

    @GetMapping
    public ResponseEntity<List<Payroll>> getAllPayrolls() {
        return ResponseEntity.ok(payrollService.getAllPayrolls());
    }

    @GetMapping("/{id}")
    public ResponseEntity<Payroll> getPayrollById(@PathVariable Long id) {
        return payrollService.getPayrollById(id)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    }

    @GetMapping("/employee/{employeeId}")
    public ResponseEntity<List<Payroll>> getPayrollsByEmployee(@PathVariable String employeeId) { // Changed from Long to String
        return ResponseEntity.ok(payrollService.getPayrollsByEmployee(employeeId));
    }

    @GetMapping("/month/{month}")
    public ResponseEntity<List<Payroll>> getPayrollsByMonth(@PathVariable String month) {
        try {
            YearMonth yearMonth = YearMonth.parse(month);
            return ResponseEntity.ok(payrollService.getPayrollsByMonth(yearMonth));
        } catch (DateTimeParseException e) {
            return ResponseEntity.badRequest().build();
        }
    }

    @GetMapping("/employee/{employeeId}/month/{month}")
    public ResponseEntity<Payroll> getPayrollByEmployeeAndMonth(
        @PathVariable String employeeId, // Changed from Long to String
        @PathVariable String month) {
        try {
            YearMonth yearMonth = YearMonth.parse(month);

```

```

        return payrollService.getPayrollByEmployeeAndMonth(employeeId, yearMonth)
            .map(ResponseEntity::ok)
            .orElse(ResponseEntity.notFound().build());
    } catch (DateTimeParseException e) {
        return ResponseEntity.badRequest().build();
    }
}

@PostMapping
public ResponseEntity<?> createPayroll(@RequestBody Map<String, Object> payrollRequest) {
    System.out.println("Incoming payroll request: " + payrollRequest);
    try {
        // Validate required fields
        if (!payrollRequest.containsKey("empId") || !payrollRequest.containsKey("month") ||
            !payrollRequest.containsKey("base") || !payrollRequest.containsKey("amount")) {
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(Map.of("error", "Missing required fields: empId, month, base, amount"));
        }

        String employeeId = payrollRequest.get("empId").toString();
        String monthStr = (String) payrollRequest.get("month");

        // Log for debugging
        System.out.println("Looking up employee with ID: " + employeeId);
        System.out.println("Processing month: " + monthStr);

        Double base = Double.valueOf(payrollRequest.get("base").toString());
        Double bonus = Double.valueOf(payrollRequest.get("bonus").toString());
        Double deductions = Double.valueOf(payrollRequest.get("deductions").toString());
        Double amount = Double.valueOf(payrollRequest.get("amount").toString());
        String notes = (String) payrollRequest.get("notes");

        // Validate employee exists
        Optional<Employee> employeeOpt = employeeService.getEmployeeById(employeeId);
        if (employeeOpt.isEmpty()) {
            System.out.println("Employee not found with ID: " + employeeId);
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(Map.of("error", "Employee with ID " + employeeId + " not found"));
        }
        System.out.println("Employee found: " + employeeOpt.get().getName());

        // Parse month with better error handling
        YearMonth month;
        try {
            month = YearMonth.parse(monthStr);
        } catch (DateTimeParseException e) {
            System.out.println("Invalid month format: " + monthStr);
            return ResponseEntity.status(HttpStatus.BAD_REQUEST)
                .body(Map.of("error", "Invalid month format. Expected yyyy-MM format"));
        }

        Optional<Payroll> existingPayroll =
            payrollService.getPayrollByEmployeeAndMonth(employeeId, month);
    }
}

```

```

    if (existingPayroll.isPresent()) {
        System.out.println("Payroll already exists for employee " + employeeId + " and month " +
month);
        return ResponseEntity.status(HttpStatus.CONFLICT)
            .body(Map.of("error", "Payroll already exists for this employee and month"));
    }

    Payroll payroll = new Payroll();
    payroll.setEmployee(employeeOpt.get());
    payroll.setMonth(month);
    payroll.setBase(base);
    // Make sure baseSalary is also set - this is for the database field
    if (payroll.getClass().getDeclaredFields() != null &&
        java.util.Arrays.stream(payroll.getClass().getDeclaredFields())
            .anyMatch(field -> field.getName().equals("baseSalary"))) {
        try {
            java.lang.reflect.Field field = payroll.getClass().getDeclaredField("baseSalary");
            field.setAccessible(true);
            field.set(payroll, base);
        } catch (Exception e) {
            System.out.println("Could not set baseSalary field: " + e.getMessage());
        }
    }
    payroll.setBonus(bonus);
    payroll.setDeductions(deductions);
    payroll.setAmount(amount);
    payroll.setNotes(notes);
    payroll.setCreatedDate(LocalDate.now());

    System.out.println("Saving new payroll: " + payroll);
    Payroll savedPayroll = payrollService.savePayroll(payroll);
    System.out.println("Payroll saved successfully with ID: " + savedPayroll.getId());
    return ResponseEntity.status(HttpStatus.CREATED).body(savedPayroll);

} catch (NumberFormatException e) {
    System.out.println("Number format error: " + e.getMessage());
    return ResponseEntity.status(HttpStatus.BAD_REQUEST)
        .body(Map.of("error", "Invalid number format: " + e.getMessage()));
} catch (Exception e) {
    e.printStackTrace();
    System.out.println("Unexpected error creating payroll: " + e.getMessage());
    return ResponseEntity.status(HttpStatus.INTERNAL_SERVER_ERROR)
        .body(Map.of("error", "Error creating payroll: " + e.getMessage()));
}
}

@PutMapping("/{id}")
public ResponseEntity<Payroll> updatePayroll(
    @PathVariable Long id,
    @RequestBody Map<String, Object> payrollRequest) {
    try {
        Optional<Payroll> existingPayrollOpt = payrollService.getPayrollById(id);
        if (existingPayrollOpt.isEmpty()) {

```

```

        return ResponseEntity.notFound().build();
    }

    Payroll existingPayroll = existingPayrollOpt.get();

    if (payrollRequest.containsKey("base")) {
        existingPayroll.setBase(Double.valueOf(payrollRequest.get("base").toString()));
    }

    if (payrollRequest.containsKey("bonus")) {
        existingPayroll.setBonus(Double.valueOf(payrollRequest.get("bonus").toString()));
    }

    if (payrollRequest.containsKey("deductions")) {
        existingPayroll.setDeductions(Double.valueOf(payrollRequest.get("deductions").toString()));
    }

    if (payrollRequest.containsKey("amount")) {
        existingPayroll.setAmount(Double.valueOf(payrollRequest.get("amount").toString()));
    }

    if (payrollRequest.containsKey("notes")) {
        existingPayroll.setNotes((String) payrollRequest.get("notes"));
    }

    Payroll updatedPayroll = payrollService.savePayroll(existingPayroll);
    return ResponseEntity.ok(updatedPayroll);

} catch (Exception e) {
    e.printStackTrace();
    return ResponseEntity.badRequest().build();
}

}

@DeleteMapping("/{id}")
public ResponseEntity<Void> deletePayroll(@PathVariable Long id) {
    if (payrollService.getPayrollById(id).isEmpty()) {
        return ResponseEntity.notFound().build();
    }

    payrollService.deletePayroll(id);
    return ResponseEntity.noContent().build();
}
}

```

PAYROLL REPOSITORY

```
package com.example.demopayrollpro.repository;
```

```

import java.time.YearMonth;
import java.util.List;
import java.util.Optional;

import org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.stereotype.Repository;

import com.example.demopayrollpro.model.Employee;
import com.example.demopayrollpro.model.Payroll;

@Repository
public interface PayrollRepository extends JpaRepository<Payroll, Long> {

    List<Payroll> findByEmployee(Employee employee);

    List<Payroll> findByMonth(YearMonth month);

    Optional<Payroll> findByEmployeeAndMonth(Employee employee, YearMonth month);
}

```

PAYROLL SERVICE

```

package com.example.demopayrollpro.service;

import java.time.LocalDate;
import java.time.YearMonth;
import java.util.List;
import java.util.Optional;

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.stereotype.Service;

import com.example.demopayrollpro.model.Employee;
import com.example.demopayrollpro.model.Payroll;
import com.example.demopayrollpro.repository.EmployeeRepository;
import com.example.demopayrollpro.repository.PayrollRepository;

@Service
public class PayrollService {

    @Autowired
    private PayrollRepository payrollRepository;

    @Autowired
    private EmployeeRepository employeeRepository;

    public List<Payroll> getAllPayrolls() {
        return payrollRepository.findAll();
    }
}

```

```

    }

    public Optional<Payroll> getPayrollById(Long id) {
        return payrollRepository.findById(id);
    }

    public List<Payroll> getPayrollsByEmployee(String employeeId) { // Changed
from Long to String
        Optional<Employee> employee =
employeeRepository.findById(employeeId);
        return employee.map(payrollRepository::findByEmployee).orElse(List.of());
    }

    public List<Payroll> getPayrollsByMonth(YearMonth month) {
        return payrollRepository.findByMonth(month);
    }

    public Optional<Payroll> getPayrollByEmployeeAndMonth(String
employeeId, YearMonth month) { // Changed from Long to String
        Optional<Employee> employee =
employeeRepository.findById(employeeId);
        return employee.flatMap(emp ->
payrollRepository.findByEmployeeAndMonth(emp, month));
    }

    public Payroll savePayroll(Payroll payroll) {
        if (payroll.getCreatedDate() == null) {
            LocalDate now = LocalDate.now();
            payroll.setCreatedDate(now);
        }

        System.out.println("Saving payroll: " + payroll);
        return payrollRepository.save(payroll);
    }

    public void deletePayroll(Long id) {
        payrollRepository.deleteById(id);
    }
}

```

EMPLOYEE PAYROLL.CSS:

```
<style>
```

```
:root {  
  --primary: #2c3e50;  
  --secondary: #3498db;  
  --success: #2ecc71;  
  --danger: #e74c3c;  
  --light: #ecf0f1;  
  --dark: #34495e;  
}
```

```
* {  
  box-sizing: border-box;  
  margin: 0;  
  padding: 0;  
}
```

```
body {  
  font-family: "Segoe UI", Tahoma, Geneva, Verdana, sans-serif;  
  background: #f5f7fa;  
  color: #333;  
  line-height: 1.6;  
}
```

```
.container {  
  width: 100%;
```

```
max-width: 1200px;
margin: 0 auto;
padding: 0 20px;
}
```

```
.hidden {
  display: none !important;
}
```

```
.header {
  background: var(--primary);
  color: white;
  padding: 1rem 0;
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
}
```

```
.header h1 {
  margin: 0;
  font-size: 1.8rem;
}
```

```
.header-content {
  display: flex;
  justify-content: space-between;
  align-items: center;
}
```

```
.logo {
```

```
display: flex;
align-items: center;
}
```

```
.logo i {
margin-right: 10px;
font-size: 1.5rem;
}
```

```
.btn {
padding: 8px 16px;
border: none;
border-radius: 4px;
cursor: pointer;
font-weight: 600;
transition: all 0.3s ease;
}
```

```
.btn-primary {
background: var(--secondary);
color: white;
}
```

```
.btn-primary:hover {
background: #2980b9;
}
```

```
.btn-success {
```

```
background: var(--success);  
color: white;  
}
```

```
.btn-success:hover {  
background: #27ae60;  
}
```

```
.btn-danger {  
background: var(--danger);  
color: white;  
}
```

```
.btn-danger:hover {  
background: #c0392b;  
}
```

```
.btn-light {  
background: var(--light);  
color: var(--dark);  
}
```

```
.btn-light:hover {  
background: #bdc3c7;  
}
```

```
.landing-page {  
display: flex;
```

```
    justify-content: center;
    align-items: center;
    height: 100vh; /* Center vertically */
    text-align: center;
}

.landing-options {
    display: flex;
    flex-direction: column;
    gap: 20px;
    width: 300px;
    margin: 0 auto; /* Center horizontally */
}

.landing-page .container {
    display: flex;
    justify-content: center;
    align-items: center;
    width: 100%;
}

.landing-options button {
    padding: 15px 20px;
    font-size: 1.2rem;
}

.auth-form {
    max-width: 400px;
```

```
margin: 0 auto; /* Center horizontally */
padding: 30px;
background: white;
border-radius: 8px;
box-shadow: 0 4px 10px rgba(0, 0, 0, 0.1);
display: flex; /* Use flexbox */
flex-direction: column; /* Stack children vertically */
justify-content: center; /* Center children vertically */
}
```

/* Add these styles to center the admin and employee pages vertically */

```
#admin-page,
#employee-page {
  height: 100vh;
  display: flex;
  align-items: center;
  justify-content: center;
}
```

```
.auth-form h2 {
  margin-bottom: 20px;
  color: var(--primary);
  text-align: center;
}
```

```
.form-group {
  margin-bottom: 20px;
}
```

```
.form-group label {  
  display: block;  
  margin-bottom: 8px;  
  font-weight: 600;  
  color: var(--dark);  
}
```

```
.form-control {  
  width: 100%;  
  padding: 10px;  
  border: 1px solid #ddd;  
  border-radius: 4px;  
  font-size: 16px;  
}
```

```
.form-control:focus {  
  outline: none;  
  border-color: var(--secondary);  
  box-shadow: 0 0 0 2px rgba(52, 152, 219, 0.2);  
}
```

```
.dashboard {  
  padding: 30px 0;  
}
```

```
.dashboard-header {  
  display: flex;
```

```
    justify-content: space-between;
    align-items: center;
    margin-bottom: 30px;
}
```

```
.dashboard-title {
    color: var(--primary);
}
```

```
.admin-actions {
    display: grid;
    grid-template-columns: repeat(auto-fit, minmax(200px, 1fr));
    gap: 20px;
    margin-bottom: 30px;
}
```

```
.action-card {
    background: white;
    padding: 20px;
    border-radius: 8px;
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);
    text-align: center;
    transition: transform 0.3s ease, box-shadow 0.3s ease;
}
```

```
.action-card:hover {
    transform: translateY(-5px);
    box-shadow: 0 5px 15px rgba(0, 0, 0, 0.1);
}
```

```
}
```

```
.action-card i {  
  font-size: 2rem;  
  color: var(--secondary);  
  margin-bottom: 15px;  
}
```

```
.action-card h3 {  
  margin-bottom: 10px;  
  color: var(--primary);  
}
```

```
.dashboard-content {  
  background: white;  
  padding: 20px;  
  border-radius: 8px;  
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);  
  margin-bottom: 30px;  
}
```

```
.section-title {  
  margin-bottom: 20px;  
  padding-bottom: 10px;  
  border-bottom: 2px solid var(--light);  
  color: var(--primary);  
}
```

```
.employee-table {  
  width: 100%;  
  border-collapse: collapse;  
}  
  
.employee-table th,  
.employee-table td {  
  padding: 12px 15px;  
  text-align: left;  
  border-bottom: 1px solid #ddd;  
}  
  
.employee-table th {  
  background-color: var(--light);  
  color: var(--dark);  
  font-weight: 600;  
}  
  
.employee-table tr:hover {  
  background-color: rgba(236, 240, 241, 0.5);  
}  
  
.employee-table .actions {  
  display: flex;  
  gap: 10px;  
}  
  
.employee-table .actions button {
```

```
padding: 6px 12px;  
font-size: 0.9rem;  
}
```

```
.form-section {  
  max-width: 600px;  
  margin: 0 auto;  
}
```

```
.form-row {  
  display: flex;  
  gap: 15px;  
  margin-bottom: 15px;  
}
```

```
.form-row .form-group {  
  flex: 1;  
  margin-bottom: 0;  
}
```

```
.report-card {  
  background: white;  
  padding: 20px;  
  border-radius: 8px;  
  box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);  
  margin-bottom: 20px;  
}
```

```
.report-card h3 {  
  color: var(--primary);  
  margin-bottom: 15px;  
  padding-bottom: 10px;  
  border-bottom: 1px solid #ddd;  
}
```

```
.report-details {  
  margin-bottom: 15px;  
}
```

```
.report-row {  
  display: flex;  
  margin-bottom: 10px;  
}
```

```
.report-label {  
  width: 150px;  
  font-weight: 600;  
}
```

```
.report-value {  
  flex: 1;  
}
```

```
.payroll-history {  
  margin-top: 20px;  
}
```

```
.payroll-month {  
  font-weight: 600;  
  color: var(--secondary);  
}
```

```
.loader {  
  display: flex;  
  justify-content: center;  
  align-items: center;  
  height: 100px;  
}
```

```
.spinner {  
  border: 4px solid rgba(0, 0, 0, 0.1);  
  border-radius: 50%;  
  border-top: 4px solid var(--secondary);  
  width: 30px;  
  height: 30px;  
  animation: spin 1s linear infinite;  
}
```

```
@keyframes spin {  
  0% {  
    transform: rotate(0deg);  
  }  
  100% {  
    transform: rotate(360deg);  
  }  
}
```

```
    }  
  }  
  
  .user-dashboard {  
    padding: 30px 0;  
  }  
  
  .user-header {  
    background: var(--light);  
    padding: 20px;  
    border-radius: 8px;  
    margin-bottom: 30px;  
  }  
  
  .user-payslip {  
    background: white;  
    padding: 30px;  
    border-radius: 8px;  
    box-shadow: 0 2px 5px rgba(0, 0, 0, 0.1);  
  }  
  
  .payslip-header {  
    text-align: center;  
    margin-bottom: 30px;  
  }  
  
  .payslip-company {  
    font-size: 1.5rem;
```

```
font-weight: bold;
color: var(--primary);
margin-bottom: 10px;
}
```

```
.payslip-title {
font-size: 1.2rem;
color: var(--dark);
margin-bottom: 5px;
}
```

```
.payslip-period {
font-size: 1rem;
color: #777;
}
```

```
.payslip-details {
display: grid;
grid-template-columns: 1fr 1fr;
gap: 30px;
margin-bottom: 30px;
}
```

```
.payslip-section {
border: 1px solid #ddd;
border-radius: 8px;
padding: 15px;
}
```

```
.payslip-section h3 {  
  margin-bottom: 15px;  
  padding-bottom: 10px;  
  border-bottom: 1px solid #ddd;  
  color: var(--primary);  
}
```

```
.payslip-row {  
  display: flex;  
  justify-content: space-between;  
  margin-bottom: 10px;  
}
```

```
.payslip-label {  
  font-weight: 600;  
}
```

```
.payslip-total {  
  margin-top: 20px;  
  padding-top: 10px;  
  border-top: 2px solid #ddd;  
  font-weight: bold;  
  font-size: 1.1rem;  
}
```

```
/* Media Queries */
```

```
@media (max-width: 768px) {
```

```
.admin-actions {  
  grid-template-columns: repeat(2, 1fr);  
}
```

```
.form-row {  
  flex-direction: column;  
  gap: 0;  
}
```

```
.payslip-details {  
  grid-template-columns: 1fr;  
  gap: 20px;  
}  
}
```

```
@media (max-width: 576px) {  
  .admin-actions {  
    grid-template-columns: 1fr;  
  }  
}
```

```
.employee-table {  
  font-size: 0.9rem;  
}
```

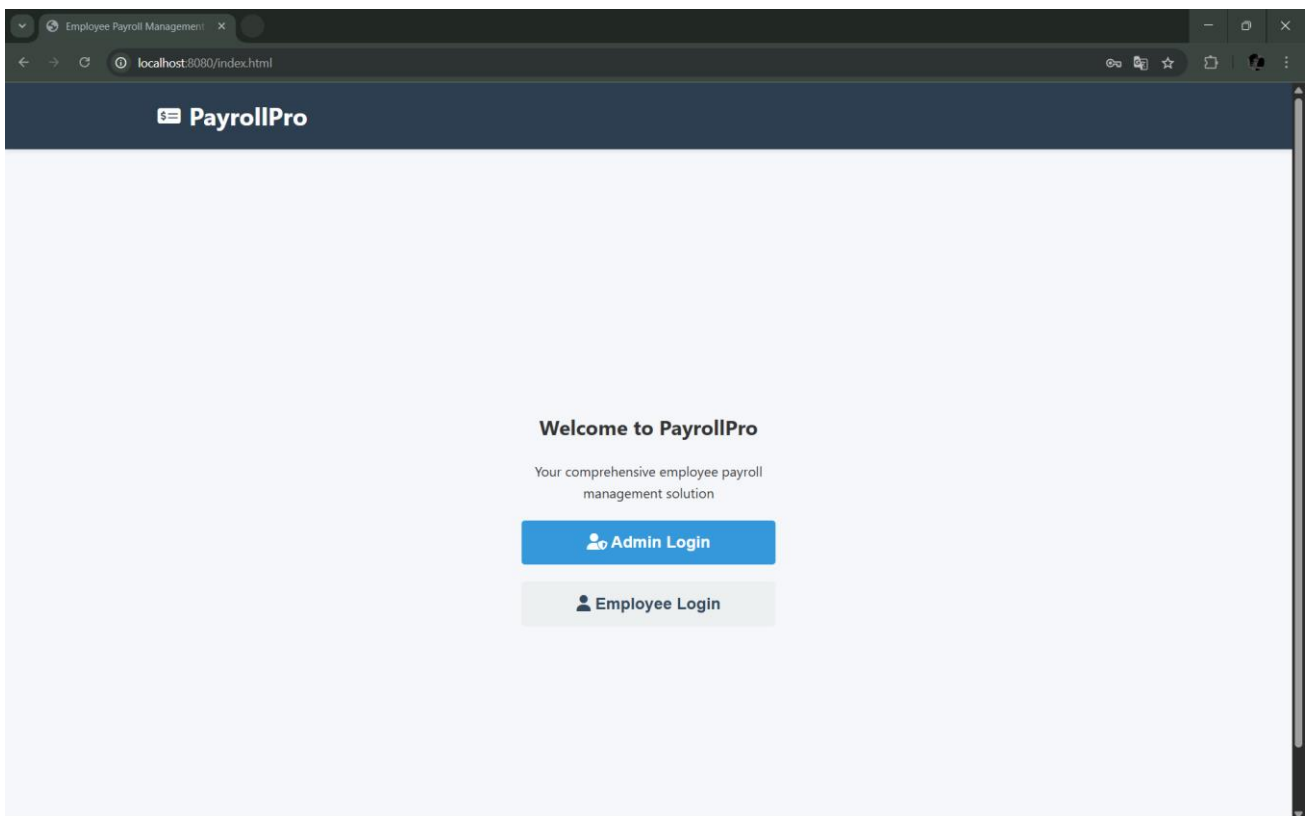
```
.employee-table th,  
.employee-table td {  
  padding: 8px;  
}
```

```
.header-content {  
  flex-direction: column;  
  gap: 10px;  
}  
  
.header h1 {  
  font-size: 1.5rem;  
}  
}  
</style>  
}
```

CHAPTER 9

SCREENSHOTS

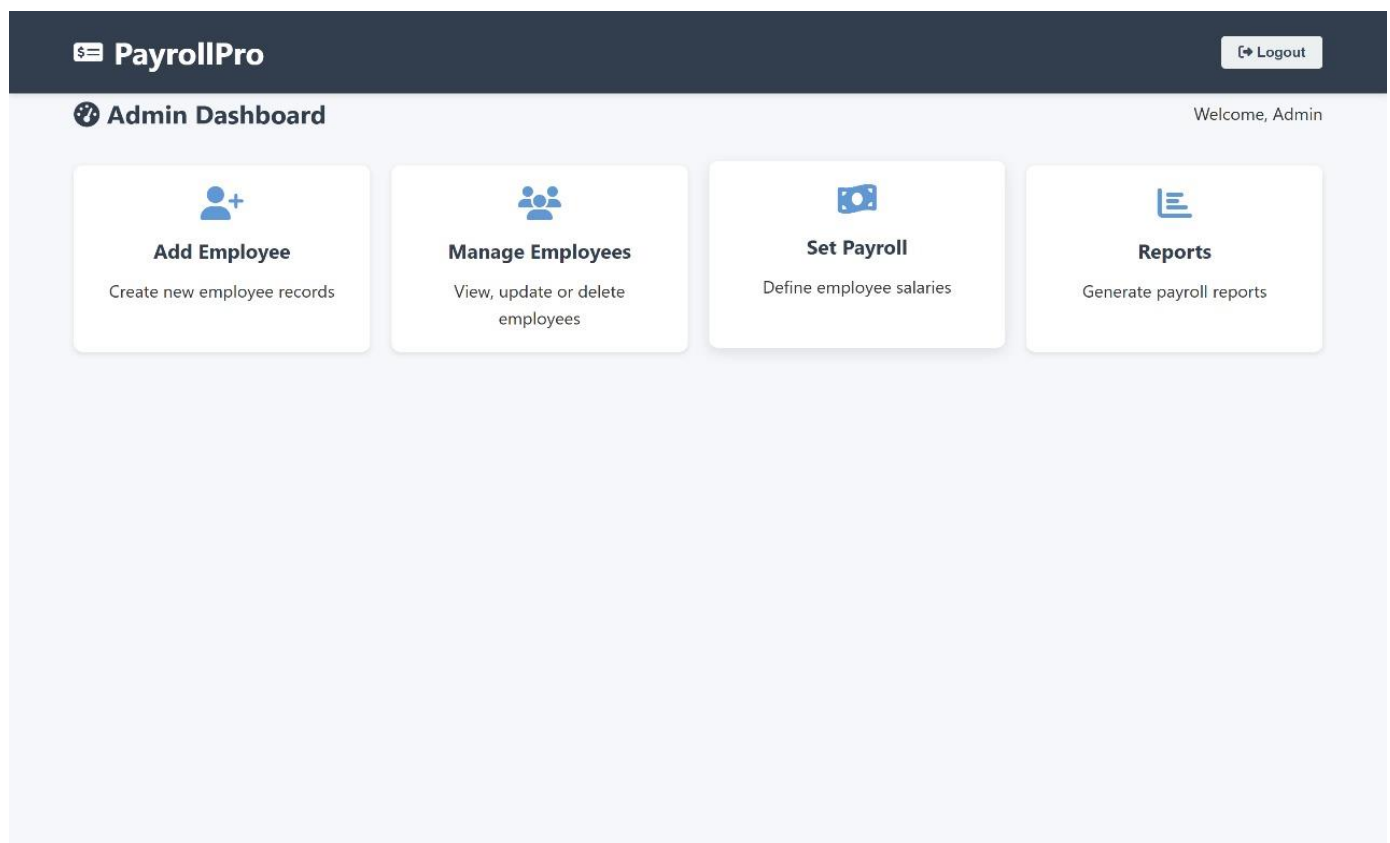
LOGIN PAGE



The landing page of the PayrollPro system provides a role-based login interface for secure access. Users can choose between Admin and Employee login options. This page ensures that only authorized personnel can access specific features of the system.

It also introduces the application as a comprehensive employee payroll management solution, setting the tone for a professional and organized system experience.

ADMIN DASHBOARD



The Admin Dashboard provides a central hub for managing all payroll system operations. It includes quick-access cards for key functions such as adding new employees, managing existing employee records, setting payroll details, and generating payroll reports.

ADD EMPLOYEE (ADMIN VIEW)

Add Employee
Create new employee records

Manage Employees
View, update or delete employees

Set Payroll
Define employee salaries

Reports
Generate payroll reports

Add/Update Employee

Employee ID	Full Name
212	Sung jin woo
Email	Phone
sung@gmail.com	9334238419
Position	Department
HR	IT
Join Date	Base Salary
16-12-2024	100000
Default Password	
employee123	

Employee can change this after first login

CancelSave Employee

Add Employee
Create new employee records

Manage Employees
View, update or delete employees

Set Payroll
Define employee salaries

Reports
Generate payroll reports

Add/Update Employee

Employee ID	Full Name
212	Sung jin woo
Email	Phone
sung@gmail.com	9334238419
Position	Department
HR	IT
Join Date	Base Salary
16-12-2024	100000
Default Password	
employee123	

Employee can change this after first login

CancelSave Employee

localhost:8080 says

Employee added successfully!

OK

The Admin can add a new employee by filling in a detailed form that includes fields such as Employee ID, Full Name, Email, Phone Number, Department, Position, Join Date, Base Salary, and a default password. Once the employee is successfully added, the system displays a confirmation and updates the employee list on the interface. The added employees appear in a structured table format showing key details, enabling the Admin to view, update, or delete employee records efficiently. This process ensures accurate record-keeping and easy management of employee data.

MANAGE PAYROLL

PayrollPro

Logout

Admin Dashboard

Welcome, Admin

Add Employee

Create new employee records

Manage Employees

View, update or delete employees

Set Payroll

Define employee salaries

Reports

Generate payroll reports

Employee Management

Search employees...

ID	Name	Position	Department	Salary	Actions
212	Sung jin woo	HR	IT	₹100000.00	
28	pradeep	Clerk	IT	₹5000.00	
EMP001	John Doe	Manager	IT	₹75000.00	
EMP002	Jane Smith	HR	Human Resources	₹65000.00	

The **Manage Employees** page provides a detailed list of all employees added to the system, displaying key information such as ID, name, contact, department, and position. It allows the Admin to view, update, or delete employee records through quick action buttons, making employee management simple, efficient, and organized.

50

SET PAYROLL

**Add Employee**
Create new employee records

**Manage Employees**
View, update or delete employees

**Set Payroll**
Define employee salaries

**Reports**
Generate payroll reports

Set Payroll

Select Employee
Sung jin woo (212) ▼

Month
May, 2025 📅

Base Salary
100000

Bonus
10000

Deductions

Net Amount
110000.00

Notes
Any additional notes...

Cancel

Save Payroll

**Add Employee**
Create new employee records

**M**
View, update or delete employees

**Set Payroll**
Define employee salaries

**Reports**
Generate payroll reports

Set Payroll

Select Employee
Sung jin woo (212) ▼

Month
May, 2025 📅

Base Salary
100000

Bonus
10000

Deductions

Net Amount
110000.00

Notes
Any additional notes...

Cancel

Save Payroll

localhost:8080 says
Payroll saved successfully!
OK

The Set Payroll page allows the Admin to define and manage monthly salary details for each employee. The Admin selects the employee and month, then enters values such as base salary, bonus, deductions, and optional notes. The system automatically calculates the net amount to be paid. This streamlined interface ensures accurate payroll generation and efficient financial record management for every employee.

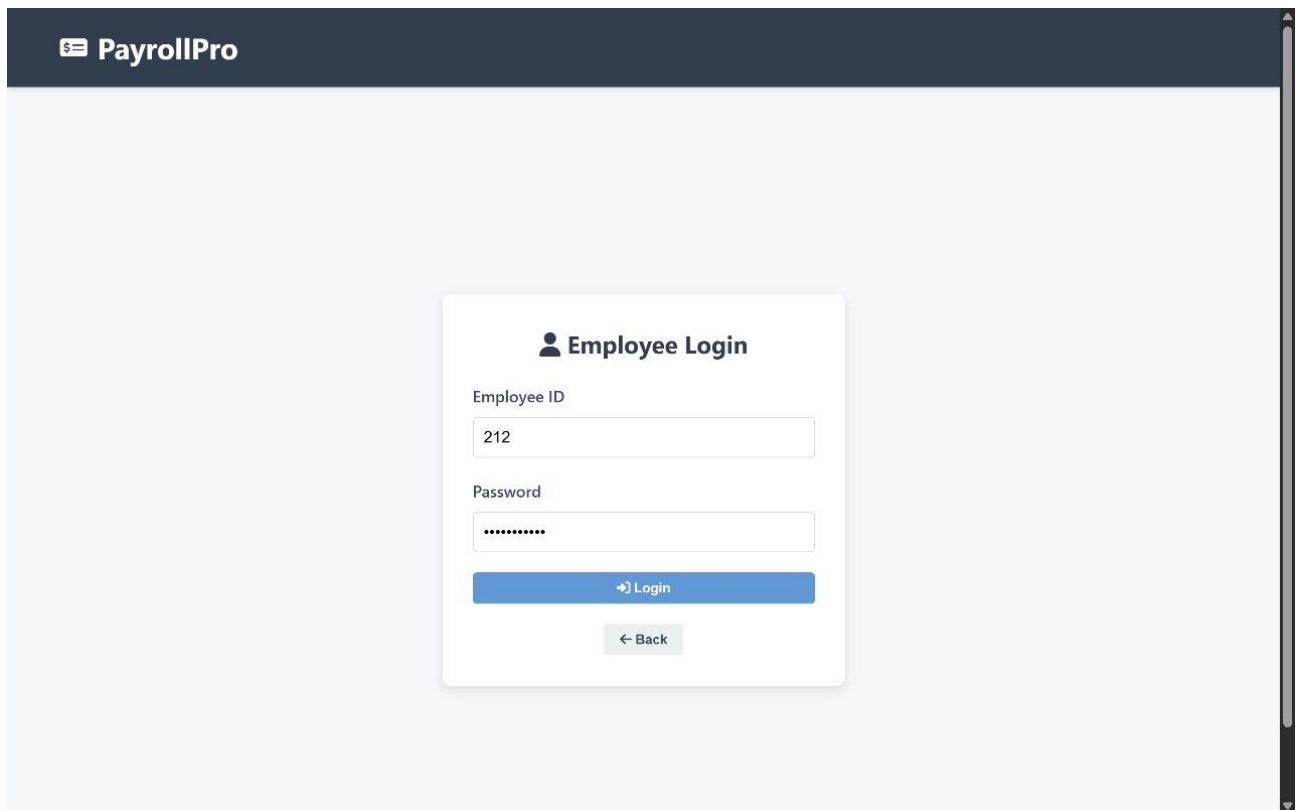
PAYROLL REPORT STATUS (ADMIN VIEW)

The screenshot shows the 'Payroll Reports' section of an application. At the top, there's a tab labeled 'employees'. Below it, the 'Payroll Reports' header is followed by two dropdown menus: 'Select Employee' (showing 'Sung jin woo (212)') and 'Month' (showing 'May, 2025'). A blue 'Generate Report' button is positioned to the right of the month dropdown. Below these is a 'Payroll Report' section containing a table of employee details. The table lists: Employee Name (Sung jin woo), Position (HR), Department (IT), Month (May 2025), Base Salary (₹100000.00), Bonus (₹10000.00), Deductions (₹0.00), and Net Salary (110000.00). At the bottom right of the report section are two buttons: 'Update Payroll' (green) and 'Print Report' (blue).

Payroll Report	
Employee Name:	Sung jin woo
Position:	HR
Department:	IT
Month:	May 2025
Base Salary:	₹100000.00
Bonus:	₹10000.00
Deductions:	₹0.00
Net Salary:	110000.00

The **Payroll Reports** page enables the Admin to generate detailed salary reports for any employee based on the selected month. It displays essential payroll data including the employee's name, department, position, base salary, bonus, deductions, and the final net salary. The report can be reviewed, updated, or printed, providing a transparent and professional overview of payroll records for both administrative use and official documentation.

PAYROLL REPORT STATUS (ADMIN VIEW)



The screenshot shows a web application interface for PayrollPro. At the top, there is a dark blue header bar with the PayrollPro logo on the left. The main content area has a light blue background. In the center, there is a white login form titled "Employee Login" with a person icon. The form contains two input fields: "Employee ID" with the value "212" and "Password" with masked characters "*****". Below the password field are two buttons: a blue "Login" button with a right-pointing arrow and a grey "Back" button with a left-pointing arrow.

This page represents the **Employee Login** screen, where individual employees can securely access their payroll details. The interface prompts the user to enter their unique Employee ID and password.

Upon successful authentication, employees are redirected to their personal dashboard where they can view their payroll records.

EMPLOYEE & PAYROLL REPORT STATUS (EMPLOYEE VIEW)

PayrollPro

Logout

Welcome, Sung jin woo
Employee ID: 212

PayrollPro Inc.

Salary Slip
May 2025

Employee Details	
Employee ID:	212
Name:	Sung jin woo
Position:	HR
Department:	IT

Payment Details	
Basic Salary:	₹100000.00
Bonus:	₹10000.00
Deductions:	₹0.00
Net Salary:	₹110000.00

Print Payslip

← Previous Month

→ Next Month

This page displays the **Employee Salary Slip** for a logged-in employee. It provides a clear and organized summary of the employee's personal and payment details for the selected month. On the left, it shows the employee's ID, name, position, and department. On the right, it presents salary components including basic salary, bonus, deductions, and the final net salary. The interface also includes navigation to browse previous or next month's salary slips and an option to print the payslip for official records.

EMPLOYEE DETAILS

Result Grid									
Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:			
	id	created_at	department	email	join_date	name	password	phone	position
	212	2025-05-10 06:01:27.423936	IT	sung@gmail.com	2024-12-16	Sung jin woo	employee123	9334238419	HR
	28	2025-05-10 05:56:09.334765	IT	pradeep@gmail.com	2025-05-08	pradeep	employee123	3984529421	Clerk
	EMP001	2025-05-10 05:54:22.633687	IT	john@example.com	2023-01-15	John Doe	employee123	1234567890	Manager
	EMP002	2025-05-10 05:54:22.570772	Human Resources	jane@example.com	2023-02-10	Jane Smith	employee123	9876543210	HR
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

This image displays the **Employee Table** from the project's backend database using a MySQL interface. It shows records of all employees currently stored in the system, including key details such as employee ID, name, email, department, position, join date, phone number, and password. Each row represents a unique employee entry, with timestamps indicating when the record was created. This structured table is essential for managing and retrieving employee information efficiently within the Employee Payroll Management System.

USER

Result Grid											
Filter Rows:		Edit:		Export/Import:		Wrap Cell Content:					
	id	amount	base	base_salary	bonus	created_at	created_date	deductions	month	notes	emp_id
	1	11	5000	5000	10	2025-05-10	2025-05-10	4999	2025-05	nothing personal...	28
	2	110000	100000	100000	10000	2025-05-10	2025-05-10	0	2025-05		212
	3	80000	75000	75000	5000	2025-05-10	2025-05-10	0	2025-05		EMP001
	4	69000	65000	65000	4000	2025-05-10	2025-05-10	0	2025-05		EMP002
*	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL	NULL

The table displays salary-related data for employees for the month of May 2025. It includes columns such as id, amount, base, base_salary, bonus, created_at, created_date, deductions, month, notes, and emp_id. Each row represents an employee's payment details, including bonuses and deductions. The emp_id indicates the employee reference, while the amount column reflects the total salary after adding bonuses and subtracting deductions. All records were created on 2025-05-10.

CHAPTER 10

CONCLUSION

In conclusion, the Payroll Management System provides a comprehensive and efficient solution for managing employee payrolls within an organization. By integrating employee data with payroll calculations, the system simplifies the process of salary processing, ensuring accurate and timely payments. With role-based access for Admins and a user-friendly interface for employees, the system enhances transparency and accountability. Admins can efficiently manage employee records, payroll details, and payment history, while employees can access their pay-related information securely. The system reduces manual errors, improves overall workflow efficiency, and ensures compliance with financial regulations. Overall, it is a valuable tool for organizations to streamline payroll management, improve data accuracy, and save time.